

ID	Name	Mnemonic	RTL Description	Exec Unit	Exec Pipe	instruction latency
1	Add Word	a rt, ra, rb	for j=0 to 15 by4: RT[4j+3:4j] <- RA[4j+3:4j] + RB[4j+3:4j]		1 even	3
2	Add Word Immediate	ai rt, ra, imm	s<- RepLeftBit(110,16); for j=0 to 15 by4: RT[4j+3:4j] <- RA[4j+3:4j] + s		1 even	3
3	Add Extended	addx rt,ra,rb	for i=0 to 3 by 1: cin <- RT[i*32]; {cout,sum} <- RA[i*32+31:i*32] + RB[i*32+31:i*32] + cin; RT[i*32+31:i*32] <- sum; RT[i*32] <- cout		1 even	3
4	Subtract from Word	sf rt, ra, rb	for j=0 to 15 by4: RT[4j+3:4j] <- RA[4j+3:4j] - RB[4j+3:4j]		1 even	3
5	Subtract from Word Immediate	sfi rt, ra, imm	s<- RepLeftBit(110,16); for j=0 to 15 by4: RT[4j+3:4j] <- s - RA[4j+3:4j]		1 even	3
6	Subtract from Extended	sfx rt,ra,rb	for i=0 to 3 by 1: bin <- RT[i*32]; {bout,diff} <- RA[i*32+31:i*32] - RB[i*32+31:i*32] - bin; RT[i*32+31:i*32] <- diff; RT[i*32] <- bout		1 even	3
7	Add Halfword	ah rt, ra, rb	for j=0 to 15 by2: RT[8j+7:8j] <- RA[8j+7:8j] + RB[8j+7:8j]		1 even	3
8	Add Halfword Immediate	ahi rt, ra, imm	s<- RepLeftBit(110,16); RT[8j+7:8j] <- RA[8j+7:8j] + s		1 even	3
9	Subtract from Halfword	sfh rt, ra, rb	for j=0 to 15 by2: for j=0 to 15 by2: RT[8j+7:8j] <- RA[8j+7:8j] + RB[8j+7:8j]		1 even	3
10	Subtract from Halfword Immediate	sphi rt, ra, imm	s<- RepLeftBit(110,16); for j=0 to 15 by2:RT[8j+7:8j] <- RA[8j+7:8j] + s		1 even	3
11	Carry Generate	cg rt,ra,rb	for i=0 to 3 by 1: {c,sum} <- RA[i*32+31:i*32] + RB[i*32+31:i*32]; RT[i*32+31:i*32] <- (c==1)?32'h0000_0001:32'h0000_0000		1 even	3
12	Borrow Generate	bg rt, ra, rb	for j=0 to 15 by4: RT[4j+3:4j] <- (RA[4j+3:4j] < RB[4j+3:4j]) ? 32'hFFFF_FFFF : 32'h0000_0000		1 even	3
13	AND	and rt, ra, rb	for j=0 to 15 by 1, RT[8j:8j+7] <- RA[8j:8j+7] & RB[8j:8j+7]		1 even	3
14	OR	or rt, ra, rb	for j=0 to 15 by 1, RT[8j:8j+7] <- RA[8j:8j+7] RB[8j:8j+7]		1 even	3
15	XOR	xor rt, ra, rb	for j=0 to 15 by 1, RT[8j:8j+7] <- RA[8j:8j+7] ^ RB[8j:8j+7]		1 even	3
16	NOR	nor rt, ra, rb	for j=0 to 15 by 1, RT[8j:8j+7] <- ~(RA[8j:8j+7] RB[8j:8j+7])		1 even	3
17	NAND	nand rt, ra, rb	for j=0 to 15 by 1, RT[8j:8j+7] <- ~(RA[8j:8j+7] & RB[8j:8j+7])		1 even	3
18	EQV (bitwise XNOR)	eqv rt, ra, rb	for j=0 to 15 by 1, RT[8j:8j+7] <- ~(RA[8j:8j+7] ^ RB[8j:8j+7])		1 even	3
19	AND Word Immediate	andi rt, ra, val	w <- ZeroExt(val,32); for j=0 to 15 by 4, RT[32j:32j+31] <- RA[32j:32j+31] & w		1 even	3
20	OR Word Immediate	ori rt, ra, val	w <- ZeroExt(val,32); for j=0 to 15 by 4, RT[32j:32j+31] <- RA[32j:32j+31] w		1 even	3
21	XOR Word Immediate	xori rt, ra, val	w <- ZeroExt(val,32); for j=0 to 15 by 4, RT[32j:32j+31] <- RA[32j:32j+31] ^ w		1 even	3
22	AND Halfword Immediate	andhi rt, ra, val	w <- ZeroExt(val,32); for j=0 to 15 by 2, RT[16j:16j+15] <- RA[16j:16j+15] & w		1 even	3
23	OR Halfword Immediate	orhi rt, ra, val	w <- ZeroExt(val,32); for j=0 to 15 by 2, RT[16j:16j+15] <- RA[16j:16j+15] w		1 even	3
24	XOR Halfword Immediate	xorhi rt, ra, val	w <- ZeroExt(val,32); for j=0 to 15 by 2, RT[16j:16j+15] <- RA[16j:16j+15] ^ w		1 even	3
25	Compare Equal Word	ceq rt, ra, rb	for j=0 to 15 by 4 if RA[32j:32j+31] == RB[32j:32j+31] RT[32j:32j+31] <- 32'hFFFF_FFFF else RT[32j:32j+31] <- 32'h0000_0000		1 even	3
26	Compare Equal Word Immediate	ceqi rt, ra, val	s <- signext(val) to 32-bit; for j=0 to 15 by 4 if RA[32j:32j+31] == s RT[32j:32j+31] <- 32'hFFFF_FFFF else RT[32j:32j+31] <- 32'h0000_0000		1 even	3
27	Compare Greater Than Word (signed)	cgt rt, ra, rb	for j=0 to 15 by 4 if \$signed(RA[32j:32j+31]) > \$signed(RB[32j:32j+31]) RT[32j:32j+31] <- 32'hFFFF_FFFF else RT[32j:32j+31] <- 32'h0000_0000		1 even	3
28	Compare Greater Than Word Immediate (signed)	cgthi rt, ra, val	s <- signext(val) to 32-bit; for j=0 to 15 by 4 if \$signed(RA[32j:32j+31]) > s RT[32j:32j+31] <- 32'hFFFF_FFFF else RT[32j:32j+31] <- 32'h0000_0001		1 even	3
29	Compare Logical Greater Than Word (unsigned)	clgt rt, ra, rb	for j=0 to 16 by 4 if RA[32j:32j+31] > RB[32j:32j+31] RT[32j:32j+31] <- 32'hFFFF_FFFF ; else RT[32j:32j+31] <- 32'h0000_0000		1 even	3
30	Compare Logical Greater Than Word Immediate (unsigned)	clgthi rt, ra, val	s <- zeroext(val) to 32-bit; for j=0 to 16 by 4 if RA[32j:32j+31] > s RT[32j:32j+31] <- 32'hFFFF_FFFF ; else RT[32j:32j+31] <- 32'h0000_0000		1 even	3
31	Compare Equal Halfword	ceqh rt, ra, rb	for j=0 to 15 by 2 if RA[16j:16j+15] == RB[16j:16j+15] RT[16j:16j+15] <- 16'hFFFF else RT[16j:16j+15] <- 16'h0000		1 even	3
32	Compare Equal Halfword Immediate	ceqhi rt, ra, val	s <- signext(val) to 16-bit; for j=0 to 15 by 2 if RA[16j:16j+15] == s RT[16j:16j+15] <- 16'hFFFF else RT[16j:16j+15] <- 16'h0000		1 even	3
33	Compare Greater Than Halfword Immediate (signed)	cgthi rt, ra, val	s <- signext(val) to 16-bit; for j=0 to 15 by 2 if RA[16j:16j+15] > \$signed(s) RT[16j:16j+15] <- 16'hFFFF else RT[16j:16j+15] <- 16'h0001		1 even	3
34	Count Leading Zeros (word)	clz rt,ra	for j=0 to 3 by 1: x<-RA[32j:32j+31]; RT[32j:32j+31] <- CLZ32(x)		1 even	3
35	Compare Logical Greater Than Halfword	clgth rt,ra,rb	for j=0 to 7 by 1: RT[16*j+15:16*j] <- (Unsigned(RA[16*j+15:16*j]) > Unsigned(RB[16*j+15:16*j])) ? 16'hFFFF : 16'h0000		1 even	3
36	Compare Logical Greater Than Halfword Immediate	clgthi rt,ra,val	s <- ZeroExt(val,16); for j=0 to 7 by 1: RT[16*j+15:16*j] <- (Unsigned(RA[16*j+15:16*j]) > Unsigned(s)) ? 16'hFFFF : 16'h0000		1 even	3
37	Select Bits (word)	selb rt,ra,rb	for j=0 to 3 by 1: A<-RA[32j:32j+31]; B<-RB[32j:32j+31]; M<-RT[32j:32j+31]; RT[32j:32j+31] <- (A & ~M) (B & M)		1 even	3
38	Immediate Load Halfword	ilh rt,val	h <- ZeroExt(val,16); for j=0 to 7 by 1: RT[16*j+15:16*j] <- h		1 even	3
39	Immediate Load Halfword Upper	ilhu rt,val	h <- ZeroExt(val,16); for i=0 to 3 by 1: RT[32*i+31:32*i] <- {h,16'h0000}		1 even	3
40	Immediate Load Word	il rt,val	w <- ZeroExt(val,32); for i=0 to 3 by 1: RT[32*i+31:32*i] <- w		1 even	3
41	Immediate Load Address	ila rt,val	a <- ZeroExt(val,32); for i=0 to 3 by 1: RT[32*i+31:32*i] <- a		1 even	3
42	shift left word	shl rt,ra,rb	sa <- RB[4:0]; for j=0 to 3 by 1: RT[32j:32j+31] <- (RA[32j:32j+31] << sa)		2 even	4
43	shift left word imm	shli rt,ra,val	sa <- val[4:0]; for j=0 to 3 by 1: RT[32j:32j+31] <- (RA[32j:32j+31] << sa)		2 even	4
44	shift left halfword	shlh rt,ra,rb	sa <- RB[3:0]; for j=0 to 7 by 1: RT[16j:16j+15] <- (RA[16j:16j+15] << sa)		2 even	4
45	shift left halfword imm	shlhi rt,ra,val	sa <- val[3:0]; for j=0 to 7 by 1: RT[16j:16j+15] <- (RA[16j:16j+15] << sa)		2 even	4
46	shift right arithmetic word	shra rt,ra,rb	sa <- RB[4:0]; for j=0 to 3 by 1: RT[32j:32j+31] <- (Signed(RA[32j:32j+31]) >>> sa)		2 even	4
47	Shift Right Arithmetic Word Immediate	shrai rt,ra,val	sa <- val[4:0]; for j=0 to 3 by 1: RT[32j:32j+31] <- (Signed(RA[32j:32j+31]) >>> sa)		2 even	4
48	rotate word	rot rt,ra,rb	sa <- RB[4:0]; for j=0 to 3 by 1: x<-RA[32j:32j+31]; RT[32j:32j+31] <- (x<<sa) (x>>(32-sa))		2 even	4
49	rotate word imm	roti rt,ra,val	sa <- val[4:0]; for j=0 to 3 by 1: x<-RA[32j:32j+31]; RT[32j:32j+31] <- (x<<sa) (x>>(32-sa))		2 even	4
50	rotate halfword	roth rt,ra,rb	sa <- RB[3:0]; for j=0 to 7 by 1: x<-RA[16j:16j+15]; RT[16j:16j+15] <- (x<<sa) (x>>(16-sa))		2 even	4

51	rotate halfword imm	rothi rt,ra,val	sa <- val[3:0]; for j=0 to 7 by 1: x<-RA[16j:16j+15]; RT[16j:16j+15] <- (x<<sa) (x>>(16-sa))	2	even	4
52	Floating Add (SP)	fa rt,ra,rb	for j=0 to 3 by 1: RT[32j:32j+31] <- FP32_add(RA[32j:32j+31], RB[32j:32j+31])	3	even	7
53	Floating Sub (SP)	fs rt,ra,rb	for j=0 to 3 by 1: RT[32j:32j+31] <- FP32_sub(RA[32j:32j+31], RB[32j:32j+31])	3	even	7
54	Floating Multiply (SP)	fm rt,ra,rb	for j=0 to 3 by 1: RT[32j:32j+31] <- FP32_mul(RA[32j:32j+31], RB[32j:32j+31])	3	even	7
55	Floating Multiply-Add (SP)	fma rt,ra,rb	for j=0 to 3 by 1: RT[32j:32j+31] <- FP32_fma(RA[32j:32j+31], RB[32j:32j+31], RT[32j:32j+31])	3	even	7
56	Floating Multiply-Sub (SP)	fms rt,ra,rb	for j=0 to 3 by 1: RT[32j:32j+31] <- FP32_fma(RA[32j:32j+31], RB[32j:32j+31], -RT[32j:32j+31])	3	even	7
57	Floating Multiply-Subtract (SP)	fms rt,ra,rb,rc	for j=0 to 3 by 1: RT[32*j+31:32*j] <- FP32_fms(RA[32*j+31:32*j], RB[32*j+31:32*j], RC[32*j+31:32*j]) // (RA*RB) - RC	3	even	7
58	Floating Negative Multiply-Subtract (SP)	fnms rt,ra,rb,rc	for j=0 to 3 by 1: RT[32*j+31:32*j] <- FP32_fnms(RA[32*j+31:32*j], RB[32*j+31:32*j], RC[32*j+31:32*j]) // -(RA*RB) - RC	3	even	7
59	Floating Compare Equal (SP)	fceq rt,ra,rb	for j=0 to 3 by 1: RT[32j+31:32j] <- (FP32_eq(RA[32j+31:32j], RB[32j+31:32j]) ? 32'hFFFF_FFFF : 32'h0000_0000)	3	even	7
60	Floating Compare Greater Than (SP)	fcgt rt,ra,rb	for j=0 to 3 by 1: RT[32j+31:32j] <- (FP32_gt(RA[32j+31:32j], RB[32j+31:32j]) ? 32'hFFFF_FFFF : 32'h0000_0000)	3	even	7
61	Floating Compare Magnitude Equal (SP)	fcmeq rt,ra,rb	for j=0 to 3 by 1: RT[32j+31:32j] <- (FP32_eq(Abs(RA[32j+31:32j]), Abs(RB[32j+31:32j]))) ? 32'hFFFF_FFFF : 32'h0000_0000)	3	even	7
62	Floating Compare Magnitude Greater Than (SP)	fcmggt rt,ra,rb	for j=0 to 3 by 1: RT[32j+31:32j] <- (FP32_gt(Abs(RA[32j+31:32j]), Abs(RB[32j+31:32j]))) ? 32'hFFFF_FFFF : 32'h0000_0000)	3	even	7
63	Integer Multiply (lo32)	mpy rt,ra,rb	for j=0 to 3 by 1: prod <- Signed(RA[32j:32j+31]) * Signed(RB[32j:32j+31]); RT[32j:32j+31] <- prod[31:0]	3	even	8
64	Integer Multiply Unsigned (lo32)	mpyu rt,ra,rb	for j=0 to 3 by 1: prod <- Unsigned(RA[32j:32j+31]) * Unsigned(RB[32j:32j+31]); RT[32j:32j+31] <- prod[31:0]	3	even	8
65	Integer Multiply-Add (lo32)	mpya rt,ra,rb	for j=0 to 3 by 1: prod <- Signed(RA[32j:32j+31]) * Signed(RB[32j:32j+31]); RT[32j:32j+31] <- prod[31:0] + RT[32j:32j+31]	3	even	8
66	Integer Multiply Immediate (lo32)	mpyi rt,ra,val	imm <- SignExt(val,32); for j=0 to 3 by 1: prod <- Signed(RA[32j:32j+31]) * Signed(imm); RT[32j:32j+31] <- prod[31:0]	3	even	8
67	absolute differences of bytes	absdb rt,ra,rb	for j=0 to 15 by 1: a<-RA[8j:8j+7]; b<-RB[8j:8j+7]; d<-(a>b)?(a-b):(b-a); RT[8j:8j+7]<d	4	even	4
68	average bytes	avgb rt,ra,rb	for j=0 to 15 by 1: a<-RA[8j:8j+7]; b<-RB[8j:8j+7]; RT[8j:8j+7]<-(a+b+1)>>1	4	even	4
69	sum bytes into halfwords	sumb rt,ra,rb	for j=0 to 7 by 1: a0<-RA[16j:16j+7]; a1<-RA[16j+8:16j+15]; b0<-RB[16j:16j+7]; b1<-RB[16j+8:16j+15]; RT[16j:16j+15]<-ZeroExt(a0)+ZeroExt(a1)+Zero	4	even	4
70	count ones in bytes	cntb rt,ra	for j=0 to 15 by 1: RT[8j:8j+7]<-popcount(RA[8j:8j+7])	4	even	4
71	form select mask for bytes	fsmb rt,ra	for j=0 to 15 by 1: RT[8j:8j+7]<-(RA[8j:8j+7])!=8'h00)?8'hFF:8'h00	4	even	4
72	Form Select Mask for Bytes Immediate	fsmbi rt,imm	m<-ZeroExt(imm,16); for j=0 to 15 by 1: RT[8*j+7:8*j]<-(m[j]==1)?8'hFF:8'h00	4	even	4
73	compare equal bytes	ceqb rt,ra,rb	for j=0 to 15 by 1: RT[8j:8j+7] <- (RA[8j:8j+7] == RB[8j:8j+7]) ? 8'hFF : 8'h00	4	even	4
74	compare greater-than bytes (signed)	cgtb rt,ra,rb	for j=0 to 15 by 1: RT[8j:8j+7] <- (\$signed(RA[8j:8j+7]) > \$signed(RB[8j:8j+7])) ? 8'hFF : 8'h00	4	even	4
75	compare logical greater-than bytes (unsigned)	dgtb rt,ra,rb	for j=0 to 15 by 1: RT[8j:8j+7] <- (RA[8j:8j+7] > RB[8j:8j+7]) ? 8'hFF : 8'h00	4	even	4
76	gather bits	gather rt,ra,rb	(placeholder) for j=0 to 15 by 1: RT[8j:8j+7]<RA[8*RB[4j:4j+3]:8*RB[4j:4j+3]+7]	5	odd	4
77	Shift Left Quadword by Bits	shlqbi rt,ra,rb	sa <- RB[6:0]; RT <- (RA << sa)	5	odd	4
78	Shift Left Quadword by Bits Immediate	shlqbii rt,ra,val	sa <- val[6:0]; RT <- (RA << sa)	5	odd	4
79	Shift Left Quadword by Bytes	shlqby rt,ra,rb	sa <- RB[3:0]; RT <- (RA << (8*sa))	5	odd	4
80	Shift Left Quadword by Bytes Immediate	shlqbysi rt,ra,val	sa <- val[3:0]; RT <- (RA << (8*sa))	5	odd	4
81	Rotate Quadword by Bytes	rotqby rt,ra,rb	sa <- RB[3:0]; k <- 8*sa; RT <- (RA << k) (RA >> (128-k))	5	odd	4
82	Rotate Quadword by Bytes Immediate	rotqbysi rt,ra,val	sa <- val[3:0]; k <- 8*sa; RT <- (RA << k) (RA >> (128-k))	5	odd	4
83	Rotate Quadword by Bits	rotqbi rt,ra,rb	sa <- RB[6:0]; k <- sa; RT <- (RA << k) (RA >> (128-k))	5	odd	4
84	Rotate Quadword by Bits Immediate	rotqbii rt,ra,val	sa <- val[6:0]; k <- sa; RT <- (RA << k) (RA >> (128-k))	5	odd	4
85	load quadword (d-form)	lqd rt,ra,imm	addr<-RA+(SignExt(imm,32)<<4); RT<-LS[addr]	6	odd	7
86	load quadword (a-form)	lqa rt,imm	addr<-Align16(SignExt(imm,32)); RT<-LS[addr]	6	odd	7
87	store quadword (d-form)	stqd rt,ra,imm	addr<-RA+(SignExt(imm,32)<<4); LS[addr]<-RT	6	odd	7
88	store quadword (a-form)	stqa rt,imm	addr<-Align16(SignExt(imm,32)); LS[addr]<-RT	6	odd	7
89	branch relative	br imm	PC<-PC+(SignExt(imm,32)<<2)	7	odd	2
90	branch absolute	bra ra	PC<-RA	7	odd	2
91	branch relative and set link	brsl imm	LR<-PC+4; PC<-PC+(SignExt(imm,32)<<2)	7	odd	2
92	branch absolute and set link	brasl ra	LR<-PC+4; PC<-RA	7	odd	2
93	branch if zero word	brz ra,imm	if(RA==0) PC<-PC+(SignExt(imm,32)<<2) else PC<-PC+4	7	odd	2
94	branch if not zero word	brnz ra,imm	if(RA!=0) PC<-PC+(SignExt(imm,32)<<2) else PC<-PC+4	7	odd	2
95	branch if zero halfword	brhz ra,imm	if(RA[15:0]==0) PC<-PC+(SignExt(imm,32)<<2) else PC<-PC+4	7	odd	2
96	branch if not zero halfword	brhnz ra,imm	if(RA[15:0]!=0) PC<-PC+(SignExt(imm,32)<<2) else PC<-PC+4	7	odd	2
97	branch indirect	bi ra	PC<-RA	7	odd	2
98	branch indirect if zero	biz ra,rb	if(RB==0) PC<-RA else PC<-PC+4	7	odd	2
99	branch indirect if not zero	binz ra,rb	if(RB!=0) PC<-RA else PC<-PC+4	7	odd	2
100	stop	stop	halt execution (simulation stops or sets done flag)	0	N/A	--
101	no-op (load pipe)	Inop	no state change	0	odd	--

102	no-op (execute pipe)	nop	no state change	0	even	--
0	hw_nop	nop	no state change	0	both	--